Hochschule Zittau/Görlitz
UNIVERSITY OF APPLIED SCIENCES

ES Modules & Bundling

Web Engineering 1 (319300)

Vorlesung 05

M.Sc. Christopher-Manuel Hilgner

Hochschule Zittau/Görlitz

christopher.hilgner@stud.hszg.de

2026-11-10

Agenda

— Agenda

ES Modules

Welches Problem wird gelöst?

- Bevor es Module gab, teilte sich jedes `<script>` einen globalen Namespace
- Der einzige Weg für private Werte war: Code in IIFEs wrappen
- „Dependency Management“ war damals: Manuell `<script>`-Tags im HTML ordnen
- Eine Datei konnte nicht deklarieren, welche Abhängigkeiten sie braucht

Statische, analysierbare Abhängigkeiten

- `import`/`export` sind syntaktische Deklarationen, keine runtime Funktionen (wie `require()`)
- Tools können den ganzen Dependency-Graph lesen ohne Code auszuführen

Ermöglicht folgende Features:

- **Frühes Finden von Errors:** Ein fehlendes Modul wird bei *load time* fehlschlagen, nicht wärend der Ausführung
- **Tree-shaking:** Bundlers können unerreichbare Abhängigkeiten finden und löschen
- **Optimiertes Laden:** Preloading (`modulepreload`), paralleles fetch, deferred execution

Deferred Execution:

- Klassische `<script>`-Tags ohne Attribute werden synchron ausgeführt (HTML parsing stoppt)
- Bei deferred:
  - Script Download passiert parallel
  - Script wird erst ausgeführt, wenn HTML geparst wurde
  - Ausgührungsreihenfolge von Scripts bleibt erhalten: anders als `async`, deferred Scripts werden in Document Reihenfolge ausgeführt

Isolation zwischen Modulen

- Jedes Modul hat seinen eigenen Scope
- Keine globalen Variablen (weniger Fehlerpotential)
- Keine Probleme zwischen unterschiedlichen Libraries
- Top-Level Code wird genau einmal pro URL ausgeführt, egal wie viele Importers es gibt

Live Bindings mit direktionaler Mutation

- Imports bringen read-only Zugriff auf Variablen
- Nur das besitzende Modul kann Werte ändern
- Änderungen am State propagieren nach Draußen → Flow von Daten wird verfolgbar

Warum?

Standard für die Sprache

Warum?

Robust

Ziele

ES Modules
— Ziele

1. **Construction:** `fetch` + `parse` für jedes Module (ohne Ausführung), Extrahierung von `import` Spezifikation, Rekursives Bauen eines Graphs, Errors tauchen früh auf. `<link rel="modulepreload">` erlaubt Browser parallele Discovery
2. **Instantiation:** Zuerst auf Tiefe, Ein Memory-Slot pro exportiertem Binding, Jeder Import wird zu dem gleichen Slot verlinkt. Function wird hier inizialisiert (hoisting)
3. **Evaluation:** Post-Order (Dependencies zuerst, danach Dependents), Jeder body wird einmal pro URL ausgeführt, danach dependents